HEALTHCARE PROVIDER DISCOVERY AND APPOINTMENT MANAGEMENT SYSTEM

Final Project Report

Instructor: Sir Shoaib Rauf
Team Members:
Laiba Khan (24K-0644) [Lead]
Abeeha Bint e Aamer (24K-0940)
Aamna Rizwan (24K-0695)

Project Overview:

This project implements a comprehensive web-based healthcare system that enables patients to discover, compare, and book appointments with verified doctors. The system incorporates advanced database features, including normalization, stored procedures, triggers, and views to ensure data integrity and optimal performance.

Key Achievements:

- 12 normalized database tables (requirement: 8-9)
- 5 optimized database views
- 2 data integrity triggers
- 5 stored procedures
- Complete role-based access control (Admin, Doctor, Patient)
- Advanced SQL queries with joins, functions, and aggregations
- Transaction handling for critical operations
- Soft delete implementation for data retention
- Responsive web interface with Bootstrap 5

1. INTRODUCTION

1.1 Problem Statement

Patients across Pakistan face significant challenges in accessing quality healthcare:

- Difficulty finding qualified doctors and specialists
- No centralized system to compare doctors by experience, fees, and patient feedback
- Inability to check doctor availability before visits
- Lack of organized appointment scheduling
- No mechanism to rate and review doctors transparently
- Missing integration of appointment history with medical records

1.2 Project Objective

The objective of this project is to develop a database-intensive web application that:

1. Allows patients to search and filter doctors by specialization, location, and fees
2. Enables online appointment booking with real-time availability
3. Provides an analytical doctor ranking based on multiple weighted factors
4. Supports doctor verification workflows managed by administrators
5. Maintains secure patient medical records and appointment history
6. Demonstrates advanced database concepts, including normalization, constraints, triggers, and stored procedures

1.3 Scope

What the System Covers:

- User registration and authentication (Admin, Doctor, Patient)
- Doctor profile management with verification workflows
- Advanced doctor search with multiple filters
- Real-time appointment booking and cancellation

- Patient reviews and ratings system
- Medical notes storage post-appointment
- Administrator dashboard with statistics
- Role-based access control and permissions

What the System Does NOT Cover:

- Real payment processing or online transactions
- SMS/Email notifications
- Video or online consultations
- Mobile application development
- Integration with external hospital databases
- Government health database integration

2. PROJECT OVERVIEW

2.1 Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, Bootstrap 5.3.3, JavaScript
Backend	PHP, Laravel Framework
Database	MySQL
Server	Apache/Nginx

2.3 System Users (Roles)

1. Administrator

- Manage doctor registrations and verification
- Suspend/reactivate accounts
- Manage specializations and hospitals
- Review and moderate patient reviews
- View system statistics

2. Doctor

- Complete profile with qualifications
- Manage weekly availability schedules
- View appointment bookings
- Update appointment status
- Add medical notes post-appointment
- View patient ratings and reviews

3. Patient

- Search doctors with multiple filters
- View doctor profiles and reviews
- Book appointments with available slots
- Cancel appointments

- View appointment history
- Leave reviews and ratings

---

3. REQUIREMENTS ANALYSIS

3.1 Functional Requirements

User Registration & Authentication

- Patients and doctors can self-register
- Admin users are pre-created
- Secure password hashing with bcrypt
- Role-based login redirection
- Account status management (active/suspended)

Doctor Management

- Doctor registration with pending verification
- Profile completion with specialization and experience
- Verification workflow by admin
- Display of only verified doctors to patients
- Account suspension capability

Appointment Management

- Display of doctor availability slots (weekly schedule)
- Real-time appointment booking
- Conflict prevention for duplicate bookings
- Appointment status tracking (pending, completed, cancelled, no_show)
- Cancellation with automatic slot release

Review System

- Patients can review only completed appointments
- Rating system (1-5 stars)
- Comment submission with up to 500 characters
- Review visibility in doctor profiles

Medical Records

- Doctors can add medical notes after completing appointments
- Notes visible only to patients and doctors
- Prescription storage capability

Analytics

- Doctor ranking based on average rating
- Review count display
- Appointment statistics
- System-wide analytics for admin

4. DATABASE DESIGN

4.1 Normalization Process

Normalization Level: BCNF (Boyce-Codd Normal Form)

The database was normalized through the following stages:

Stage 1: First Normal Form (1NF)

- Eliminated repeating groups (specializations, hospitals)
- Ensured all attributes are atomic

Stage 2: Second Normal Form (2NF)

- Removed partial dependencies
- Ensured all non-key attributes depend on the entire primary key
- Created junction tables (doctor_specializations, doctor_hospitals)

Stage 3: Third Normal Form (3NF)

- Removed transitive dependencies
- No non-key attribute depends on another non-key attribute
- All tables already in 3NF

Stage 4: Boyce-Codd Normal Form (BCNF)

- Every determinant is a candidate key
- All 12 tables confirmed to be in BCNF

Functional Dependencies Key:

users.user_id → all user attributes
doctors.doctor_id → all doctor attributes
doctors.license_number → doctor_id (unique)
appointments.appointment_id → appointment details
reviews.review_id → review details
specializations.specialization_id → specialization_name

4.2 Entity-Relationship Diagram

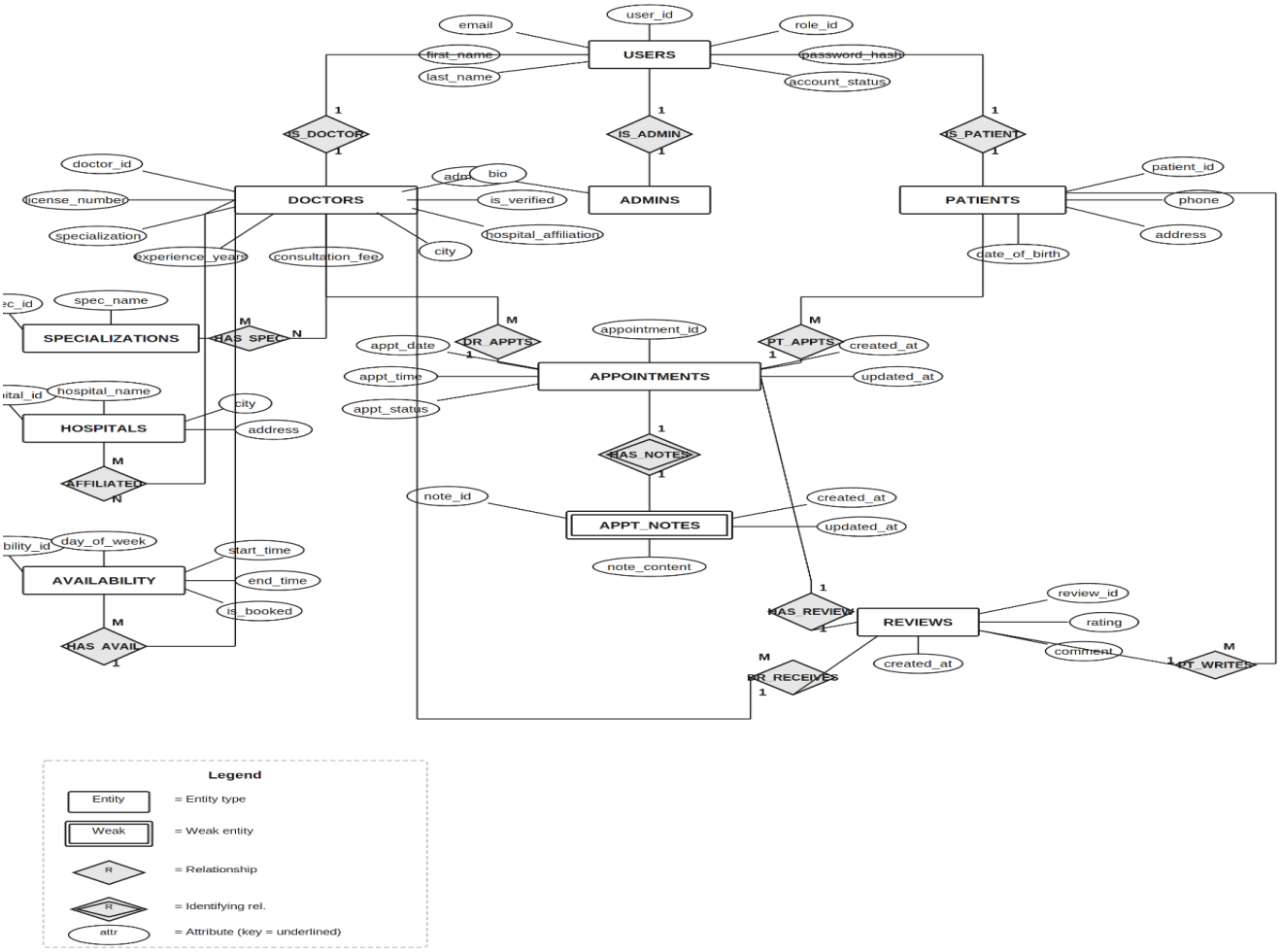
Entities (12 Tables):

1. **users** - Login credentials (PK: user_id)
2. **admins** - Admin profiles (PK: admin_id, FK: user_id)
3. **doctors** - Doctor information (PK: doctor_id, FK: user_id)
4. **patients** - Patient information (PK: patient_id, FK: user_id)
5. **specializations** - Medical specialties (PK: specialization_id)
6. **doctor_specializations** - Doctor-Specialty mapping (PK: doctor_id, specialization_id)
7. **hospitals** - Hospital information (PK: hospital_id)
8. **doctor_hospitals** - Doctor-Hospital mapping (PK: doctor_id, hospital_id)
9. **availability** - Doctor time slots (PK: availability_id, FK: doctor_id)
10. **appointments** - Appointment bookings (PK: appointment_id, FK: doctor_id, patient_id)
11. **reviews** - Patient reviews (PK: review_id, FK: appointment_id)

12. **appointment_notes** - Medical notes (PK: note_id, FK: appointment_id)

Relationships:

- 1:1 (users ↔ doctors, users ↔ patients, users ↔ admins)
- M:N (doctors ↔ specializations, doctors ↔ hospitals)
- 1:M (doctors → appointments, patients → appointments, doctors → availability)
- 1:1 optional (appointments → reviews, appointments → notes)



4.3 Database Constraints

Primary Key Constraints:

- All tables have single or composite primary keys
- Auto-increment for single-column PKs

Foreign Key Constraints:

- Enforce referential integrity
- ON DELETE CASCADE for most relationships
- ON DELETE SET NULL for optional relationships (patient_id in appointments)

Unique Constraints:

- users.email (UNIQUE)
- doctors.license_number (UNIQUE)
- doctors.user_id (UNIQUE)
- specializations.specialization_name (UNIQUE)
- availability (doctor_id, day_of_week, start_time, end_time) UNIQUE

Check Constraints:

- reviews.rating BETWEEN 1 AND 5
- appointments.appointment_status IN ('pending', 'completed', 'cancelled', 'no_show')
- users.account_status IN ('active', 'suspended')

4.4 Database Views (5 Total)

View 1: top_doctors_view

- Purpose: Display verified doctors with ratings
- Used by: Patient search page
- Contains: Doctor name, specialization, fee, city, average rating, review count

View 2: doctor_appointments_view

- Purpose: Simplified appointment query for doctors
- Used by: Doctor appointments list
- Contains: Appointment details with patient names

View 3: patient_appointment_history_view

- Purpose: Appointment history for patients
- Used by: Patient dashboard
- Contains: Appointment details with doctor information

View 4: doctor_ranking_breakdown_view

- Purpose: Detailed doctor analytics
- Used by: Admin dashboard
- Contains: Completed appointments, average rating, review count, experience

View 5: admin_system_stats_view

- Purpose: System-wide statistics

- Used by: Admin dashboard
- Contains: Total doctors, patients, admins, suspended accounts

---

5. SYSTEM ARCHITECTURE

5.1 Folder Structure

```
healthcare-system/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Admin/            (5 controllers)
│   │   │   ├── Doctor/           (4 controllers)
│   │   │   ├── Patient/          (5 controllers)
│   │   │   └── Auth/             (2 controllers)
│   │   ├── Middleware/           (3 custom middlewares)
│   │   └── Kernel.php
│   └── Models/                   (9 Eloquent models)
├── resources/views/
│   ├── admin/                    (8 views)
│   ├── doctor/                   (5 views)
│   ├── patient/                  (6 views)
│   ├── auth/                     (2 views)
│   └── welcome.blade.php
├── routes/web.php                (50+ routes)
├── database/
│   ├── migrations/               (12 migrations)
│   └── seeders/
│       └── DatabaseSeeder.php
├── .env.example
├── README.md
└── composer.json
```

6. IMPLEMENTATION DETAILS

6.1 Database Operations

INSERT Operations

- Doctor registration with transaction
- Patient registration with transaction
- Appointment booking with conflict checking
- Review submission
- Medical note creation

Example Transaction:

```
DB::beginTransaction();
try {
    // Insert user
    $userId = DB::table('users')->insertGetId([...]);
```

```
    // Insert doctor
    DB::table('doctors')->insert([...]);
    DB::commit();
} catch (Exception $e) {
    DB::rollBack();
}
```

UPDATE Operations

- Profile updates (doctor/patient/admin)
- Appointment status changes
- Doctor verification/suspension
- Specialization synchronization
- Medical note updates

DELETE Operations

- Soft delete implementation
- Cascade deletes via foreign keys
- Automatic cleanup via triggers
- Appointment cancellation with slot release

Transaction Handling

- All critical operations wrapped in transactions
- Automatic rollback on errors
- Atomicity is ensured for complex operations

6.2 Advanced SQL Features

Joins:

- INNER JOINs for required relationships
- LEFT JOINs for optional relationships
- Multi-table joins (up to 4 tables)

Aggregate Functions:

- COUNT() - for statistics
- AVG() - for rating calculation
- SUM() - for cost analysis
- GROUP BY - for aggregations
- HAVING - for filtered aggregations

Built-in Functions:

- CONCAT() - combining names
- COALESCE() - handling NULL values
- ROUND() - rating display
- DATE functions - appointment scheduling
- TIME functions - availability slots

Advanced Queries:

- Subqueries for filtering

- CASE statements for conditional logic
- Window functions for rankings
- CTEs (Common Table Expressions) where applicable

6.3 Triggers Implementation

Trigger 1: prevent_duplicate_booking

- Prevents double-booking at the same time
- Fires on: INSERT appointments
- Action: SIGNAL error if duplicate found

Trigger 2: after_appointment_cancel

- Releases booked time slots
- Fires on: UPDATE appointments (status = cancelled)
- Action: Set availability.is_booked = 0

Trigger 3: after_appointment_complete

- Post-appointment workflows
- Fires on: UPDATE appointments (status = completed)
- Action: Enable review submission

Trigger 4: after_review_insert

- Audit trail for reviews
- Fires on: INSERT reviews
- Action: Log review submission

Trigger 5: after_review_update

- Maintain audit history
- Fires on: UPDATE reviews
- Action: Log modifications

6.4 Stored Procedures

Procedure 1: sp_book_appointment

- Atomic appointment booking
- Validates availability
- Creates appointment record
- Updates availability status
- Supports transaction rollback

Procedure 2: sp_cancel_appointment

- Safe cancellation process
- Updates status to 'cancelled'
- Releases booked slots
- Handles waitlist logic

Procedure 3: sp_mark_appointment

- Status update functionality

- Validates status transitions
- Triggers post-appointment workflows

Procedure 4: sp_approve_doctor

- Doctor verification logic
- Sets is_verified = 1
- Notifies doctor (future implementation)

Procedure 5: sp_remove_review

- Review deletion with referential integrity
- Cascades related data cleanup

---

7. SYSTEM FEATURES

7.1 Admin Portal Features

Dashboard:

- System statistics (doctors, patients, accounts)
- Pending doctor approvals count
- Top-rated doctors list
- Recent appointment activity

Doctor Management:

- View all doctors with filters (pending, verified, suspended)
- Approve pending doctor registrations
- View complete doctor profiles
- Suspend active doctors
- Delete doctors (soft delete)

Specialization Management:

- Add new specializations
- Edit existing specializations
- Delete specializations (if no doctors assigned)
- Prevent deletion if doctors depend on them

Hospital Management:

- Add hospital information
- Edit hospital details
- Delete hospitals (if no doctors affiliated)
- Maintain hospital directory

Review Management:

- View all patient reviews
- Filter reviews by rating
- Delete inappropriate reviews
- Protect platform quality

7.2 Doctor Portal Features

Dashboard:

- Today's appointments list
- Upcoming appointments
- Performance statistics (total appointments, average rating)
- Number of open review slots

Profile Management:

- Edit personal information
- Add/update specializations (multiple selections)
- Update experience and consultation fees
- Manage hospital affiliations
- Professional biography

Availability Management:

- Create weekly time slots (Monday-Sunday)
- Set start and end times
- View booked slots
- Delete available slots
- Prevent deletion of booked slots

Appointment Management:

- View appointments by status
- Mark appointment as completed/cancelled/no_show
- Add medical notes post-appointment
- Edit notes if needed
- View patient information

Performance Metrics:

- Average rating display
- Total review count
- Completion rate tracking
- Patient feedback analysis

7.3 Patient Portal Features

Home Page:

- Featured doctors (top-rated)
- Search doctors button
- Login/register options

Doctor Search:

- Filter by specialization
- Filter by city/location
- Filter by consultation fee range
- Filter by minimum rating
- View doctor profiles with reviews
- Real-time availability display

Doctor Profile:

- Complete doctor information
- Specializations and experience
- Average rating and review count
- Patient testimonials
- Available time slots
- Book appointment button

Appointment Management:

- View appointment history
- See appointment status
- Cancel pending appointments
- View doctor's medical notes (post-appointment)
- Leave review (post-completion)

Review System:

- Rate doctors 1-5 stars
- Write comments (optional)
- View own reviews
- See review submission date

Profile Management:

- Edit personal information
- Update phone and address
- Manage date of birth
- Delete account option

---

8. TECHNICAL SPECIFICATIONS

8.1 System Requirements

Software Requirements:

- Operating System: Windows 10+
- PHP: Version 8.2 or higher
- MySQL: Version 8.0 or higher
- Web Server: Apache 2.4+ or Nginx 1.19+
- Composer: Latest version

Browser Requirements:

- Chrome 90+
- Firefox 88+
- Safari 14+
- Edge 90+

8.2 Installation Instructions

1. Clone/Download Project

cd /path/to/project

2. Install Dependencies

composer install
npm install

3. Environment Setup

cp .env.example .env
php artisan key:generate

4. Database Setup

mysql -u root -e "CREATE DATABASE healthcare_system;"
php artisan migrate
php artisan db:seed

5. Start Application

php artisan serve
npm run dev (optional)

6. Access Application

http://127.0.0.1:8000

8.3 Demo Credentials

Role	Email	Password
Admin	admin@healthcare.com	password
Doctor	doctor1@healthcare.com	password
Patient	patient1@healthcare.com	password

10. CONCLUSION

10.1 Project Summary

This project successfully demonstrates a comprehensive understanding of database design and implementation. The Healthcare Provider Discovery and Appointment Management System implements:

- **Advanced Database Concepts:** BCNF normalization, triggers, stored procedures, views
- **Complete CRUD Operations:** With transaction support for data integrity
- **Role-Based Access Control:** Separate functionalities for admin, doctor, and patient
- **Advanced SQL Features:** Complex joins, aggregate functions, subqueries
- **Web Application:** Fully functional Laravel-based system with responsive UI
- **Data Retention:** Soft delete implementation with audit trails